

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 01 -

PARADIGMAS DE APRENDIZADO DE MÁQUINA

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	11
MERCADO, CASES E TENDÊNCIAS	15
O QUE VOCÊ VIU NESTA AULA?	17
REFERÊNCIAS.....	19

EMAP

O QUE VEM POR AÍ?

Imagine que você é um(a) profissional de Data Science enfrentando três desafios diferentes no seu dia a dia. Primeiro, um gerente comercial pede que você desenvolva um modelo capaz de prever quais clientes estão propensos a churn (isto é, cancelar um serviço). Em seguida, a equipe de marketing quer segmentar automaticamente os usuários da plataforma em grupos com padrões de comportamento semelhantes, sem ter uma categoria pré-definida para cada um. Por fim, o time de inovação propõe que você ajude a treinar um agente inteligente para jogar um videogame e aprender a vencer o oponente por si só. Como abordar problemas tão distintos?

Cada um desses desafios corresponde a um paradigma de aprendizado de máquina (Machine Learning): o primeiro problema é um caso típico de aprendizado supervisionado, o segundo se enquadra no aprendizado não supervisionado e o terceiro envolve aprendizado por reforço. Esses três paradigmas representam as abordagens fundamentais no campo do Machine Learning, diferenciados principalmente pelo tipo de sinal ou feedback disponível para treinar o modelo.

Nesta aula, exploraremos em detalhes esses três paradigmas principais do aprendizado de máquina, discutindo seus fundamentos teóricos, técnicas e intuições associadas, além de exemplos práticos em Python para ilustrar cada abordagem. Também veremos como esses paradigmas se complementam e são aplicados em diferentes contextos, bem como as tendências atuais do mercado. Ao final, você compreenderá as diferenças, vantagens e limitações de cada paradigma e estará apto(a) a identificar qual abordagem é mais adequada para diferentes tipos de problemas em ciência de dados e inteligência artificial.

HANDS ON

Nesta seção, vamos colocar em prática cada um dos paradigmas de aprendizado de máquina. Usaremos exemplos simples em Python (com bibliotecas populares como scikit-learn e outras) para demonstrar os conceitos de aprendizado supervisionado, não supervisionado e por reforço. Todos os exemplos foram testados e são funcionais, permitindo que você os execute para reproduzir os resultados apresentados.

Exemplo de aprendizado supervisionado

```
1  from sklearn.datasets import load_iris
2  from sklearn.tree import DecisionTreeClassifier
3  from sklearn.model_selection import train_test_split
4  from sklearn.metrics import accuracy_score
5
6  # Carrega dataset de exemplo (flores Íris)
7  iris = load_iris()
8  X_train, X_test, y_train, y_test = train_test_split(
9      iris.data, iris.target, test_size=0.3,
random_state=42
10 )
11
12 # Cria e treina uma árvore de decisão
13 modelo = DecisionTreeClassifier(max_depth=3,
random_state=42)
14 modelo.fit(X_train, y_train)
15
16 # Faz previsões no conjunto de teste
17 y_pred = modelo.predict(X_test)
18
19 # Calcula a acurácia das previsões
20 acuracia = accuracy_score(y_test, y_pred)
21 print(f"Acurácia no conjunto de teste:
{acuracia:.2f}")
```

Código-fonte 1 - from sklearn.datasets import load_iris

Fonte: Elaborado pelo autor (2026)

No aprendizado supervisionado, fornecemos ao algoritmo dados de entrada e as respostas esperadas (rótulos) correspondentes, de modo que ele possa aprender a mapear corretamente as entradas em saídas. Vamos considerar um exemplo clássico de classificação: a previsão da espécie de uma flor Íris com base em

medidas de suas pétalas e sépalas. Utilizaremos o famoso conjunto de dados Iris, que contém 150 amostras de flores de três espécies de íris (Setosa, Versicolor e Virginica), cada uma descrita por quatro características (largura e comprimento de pétalas e sépalas). Nosso objetivo será treinar um modelo simples que, a partir dessas características, consiga prever a espécie da flor.

Para isso, carregaremos os dados e usaremos uma parte para treinar um modelo de classificação e outra parte para testar seu desempenho. Neste exemplo, empregaremos uma Árvore de Decisão (do módulo `sklearn.tree`) pela sua simplicidade e interpretabilidade. Em poucas linhas de código, realizaremos as etapas típicas de um problema de aprendizagem supervisionada: separar os dados em treino e teste, treinar o modelo no conjunto de treino e avaliar sua acurácia no conjunto de teste.

Ao executar o código apresentado, o modelo aprende a classificar as flores e exibe a acurácia (proporção de acertos) no conjunto de teste. Por exemplo, uma execução típica pode resultar em “Acurácia no conjunto de teste: 0.97”.

Isso significa que ~97% das amostras de teste tiveram a espécie corretamente prevista pelo modelo. Esse resultado é bastante bom, indicando que o modelo aprendeu padrões relevantes para distinguir as espécies de íris. No mundo real, técnicas supervisionadas de classificação similares podem ser usadas em aplicações como detecção de spam em e-mails, diagnóstico médico (e.g., identificar se uma imagem de raio-X indica pneumonia) ou previsão de desistência de clientes (churn).

Nesses cenários, temos um conjunto de exemplos históricos com rótulos conhecidos (e.g., quais e-mails eram spam, quais pacientes ficaram doentes, quais clientes cancelaram o serviço) e queremos que o modelo aprenda com esses exemplos para prever rótulos em novos dados.

Atenção!: Embora tenhamos obtido alta acurácia, é importante lembrar que um modelo supervisionado pode superajustar (overfit) aos dados de treinamento se não tomarmos cuidado. No nosso exemplo, usamos o método de separação treino/teste justamente para avaliar a capacidade de generalização do modelo para dados novos. Em aplicações reais, é recomendável usar técnicas como validação cruzada (cross-validation) e regularização do modelo para garantir que ele não esteja apenas memorizando os dados de treino, mas sim aprendendo padrões que se aplicam a novos exemplos. A avaliação apropriada, com dados

não vistos durante o treinamento, é fundamental para confirmar se o modelo serve ao propósito desejado.

Exemplo de aprendizado não supervisionado

No aprendizado não supervisionado, não fornecemos rótulos ou respostas corretas ao algoritmo. Em vez disso, deixamos que ele encontre por conta própria estruturas ocultas ou agrupamentos nos dados. Um caso de uso comum é o agrupamento (clustering), em que desejamos dividir um conjunto de objetos em grupos similares com base em suas características, sem saber previamente a que grupo cada objeto deveria pertencer.

Vamos demonstrar o agrupamento usando o algoritmo K-Means, uma técnica clássica de aprendizado não supervisionado para clusterização. Iremos gerar artificialmente um conjunto de pontos em duas dimensões (x e y) que formam dois agrupamentos naturais (clusters) e então usar o K-Means para ver se o algoritmo é capaz de recuperar esses agrupamentos. Confira o código a seguir:

```
1     import numpy as np
2     from sklearn.cluster import KMeans
3
4     # Gera dados de exemplo: 100 pontos em torno de dois
centros (0,0) e (3,3)
5     np.random.seed(42)
6     X = np.vstack([
7         np.random.normal(loc=[0, 0], scale=0.5,
size=(50, 2)),
8         np.random.normal(loc=[3, 3], scale=0.5,
size=(50, 2))
9     ])
10
11     # Aplica o algoritmo de agrupamento K-Means para
encontrar 2 clusters
12     kmeans = KMeans(n_clusters=2, n_init=10,
random_state=42)
13     rotulos = kmeans.fit_predict(X)
14
15     print("Quantidade de clusters encontrados:",
len(np.unique(rotulos)))
16     print("Centroides:", kmeans.cluster_centers_)
```

Código-fonte 1 – import numpy as np
Fonte: Elaborado pelo autor (2026)

No código apresentado, geramos 100 pontos distribuídos em torno de dois “centros” distintos — um próximo da coordenada (0,0) e outro próximo de (3,3). Em seguida, usamos o K-Means pedindo para formar `n_clusters=2` grupos; o algoritmo então itera realocando os pontos até convergir em uma partição dos dados. Ao executar o código, podemos obter um resultado similar a:

```
1      Quantidade de clusters encontrados: 2
2      Centroides: [[0.07 -0.01] [3.16  2.99]]
```

Código-fonte 2 – Clusters encontrados e centroides estimados pelo K-Means (Python)
Fonte: Elaborado pelo autor (2026)

Perceba que o algoritmo descobriu dois clusters distintos nos dados, sem qualquer rótulo prévio. Os centroides (centros dos grupos) estimados pelo K-Means foram aproximadamente [0.07, -0.01] e [3.16, 2.99], que estão muito próximos dos centros originais que usamos para gerar os dados ((0,0) e (3,3)). Além disso, se contarmos quantos pontos ficaram em cada cluster, veremos que cada grupo recebeu cerca de 50 pontos, alinhado com nossa geração simétrica de 50 pontos por grupo. Ou seja, o K-Means conseguiu recriar os agrupamentos originais a partir apenas dos dados não rotulados.

Esse tipo de técnica de agrupamento é bastante utilizado em diversas áreas. Por exemplo, empresas de comércio eletrônico podem segmentar clientes com base em seus padrões de compra (agrupando consumidores com preferências semelhantes), a detecção de fraudes pode ser realizada identificando anomalias que se destacam de grupos de comportamento esperado e algoritmos de compressão de dados podem descobrir estruturas latentes para reduzir a dimensionalidade dos dados (como na análise de componentes principais, PCA). Em todos esses casos, não há respostas certas pré-definidas, apenas conjuntos de dados que o algoritmo tenta organizar de modo significativo, encontrando insights ocultos que podem apoiar a tomada de decisão.

Exemplo de aprendizado por reforço

No aprendizado por reforço, o enfoque é diferente dos dois casos anteriores: aqui não temos um conjunto de dados estático para treinar um modelo com ou sem rótulos, mas sim um agente interagindo com um ambiente em uma sequência de

estados e ações, com o objetivo de maximizar recompensas ao longo do tempo. A ideia é inspirada em como aprendemos por tentativa e erro: o agente toma uma ação em determinado estado do ambiente e o ambiente responde retornando uma recompensa (um valor numérico que reflete o benefício da ação tomada) e um novo estado. A partir desse ciclo de feedback contínuo, o agente vai atualizando sua política (estratégia de como agir) para obter cada vez mais recompensa acumulada ao longo do tempo.

Vamos ilustrar um pequeno exemplo de aprendizado por reforço usando um cenário simplificado de “máquinas de caça-níquel” (multi-armed bandit). Suponha que um agente tenha 3 máquinas de caça-níquel para jogar, cada uma com uma probabilidade fixa de dar recompensa (ou seja, de ganhar algum prêmio) de 20%, 50% e 70%, respectivamente. O agente não sabe dessas probabilidades; ele precisa experimentar as diferentes máquinas e, com base nos ganhos (recompensas) obtidos, tentar descobrir qual máquina é a mais lucrativa.

Esse problema captura a essência do dilema exploração vs. exploração: o agente deve equilibrar explorar máquinas diferentes para descobrir seus retornos e explorar aquela que parece mais promissora para ganhar o máximo possível. A seguir está um código Python que simula esse cenário usando uma estratégia simples chamada epsilon-greedy (na qual, a cada rodada, com probabilidade ϵ o agente escolhe uma máquina aleatoriamente – explorando – e com probabilidade $1-\epsilon$ ele escolhe a máquina com melhor desempenho estimado até o momento – explorando o conhecimento atual):

```
1  import random
2  import numpy as np
3
4  probs = [0.2, 0.5, 0.7] # probabilidades de
recompensa de cada máquina
5  Q = [0.0, 0.0, 0.0] # estimativas iniciais de
valor (qualidade) de cada ação
6  N = [0, 0, 0] # contagem de tentativas de
cada ação
7
8  epsilon = 0.1
9  for t in range(2000):
10     # Escolhe uma ação (máquina) usando a
estratégia epsilon-greedy
11     if random.random() < epsilon:
```

```
12             acao = random.randrange(len(Q))      # explora
(escolha aleatória)
13         else:
14             acao = int(np.argmax(Q))              # explora
o melhor conhecido
15         # Executa a ação e observa recompensa (1 se
ganhar, 0 se perder)
16         recompensa = 1 if random.random() < probs[acao]
else 0
17         N[acao] += 1
18         # Atualiza a estimativa de valor Q da ação
escolhida (média incremental das recompensas)
19         Q[acao] += (recompensa - Q[acao]) / N[acao]
20
21     print("Valores Q estimados:", [round(q, 2) for q in
Q])
22     print("Melhor ação encontrada:", int(np.argmax(Q)))
```

Código-fonte 3 - Código Python com estratégia simples chamada epsilon-greedy

Fonte: Elaborado pelo autor (2026)

No código anterior, `probs` define as probabilidades reais de ganho de cada uma das 3 máquinas (20%, 50% e 70%). O agente não sabe desses valores e tenta estimá-los (Q) à medida que interage com o ambiente. A cada iteração, ele escolhe uma máquina – ocasionalmente de forma aleatória para explorar possibilidades novas, mas na maioria das vezes selecionando a máquina com maior valor Q atual para explorar o que já acredita ser melhor. Com base no retorno obtido (recompensa de 0 ou 1), ele atualiza a estimativa do valor daquela ação (calculando a média das recompensas recebidas naquela máquina até o momento). Após 2.000 rodadas dessa interação, o código imprime os valores estimados para cada máquina e qual máquina o agente considera ser a melhor:

```
1     Valores Q estimados: [0.18, 0.46, 0.71]
2     Melhor ação encontrada: 2
```

Código-fonte 4 – Valores Q estimados e melhor ação encontrada (Python)

Fonte: Elaborado pelo autor (2026)

Aqui observamos que o agente identificou corretamente a máquina de maior recompensa média, correspondendo à terceira máquina (índice 2, já que a indexação começa em 0). Os valores Q estimados aproximam-se das probabilidades reais de ganho (≈ 0.18 vs 0.2, 0.46 vs 0.5 e 0.71 vs 0.7, respectivamente), demonstrando o processo de aprendizagem por reforço: com o tempo e várias

interações, o agente aprendeu quais ações (máquinas) tendem a trazer mais retorno e passou a preferi-las.

Esse exemplo simples ilustra conceitos-chave do aprendizado por reforço, como o trade-off entre exploração e exploração (explorar novas ações vs. aproveitar o conhecimento atual) e o cálculo de valores de ação (estimativas de quão boa é cada ação em termos de recompensa esperada). Em problemas reais de reforço, como treinar um robô para andar ou um agente para jogar xadrez, o cenário é formalizado em termos de Processos de Decisão de Markov (MDP), com estados possivelmente muito numerosos ou complexos.

Nesses casos, técnicas avançadas como Deep Reinforcement Learning (Aprendizado por Reforço Profundo) são empregadas, utilizando redes neurais para estimar os valores ou políticas do agente quando o espaço de estados/ações é muito grande. Esse foi o caso, por exemplo, do sistema do DeepMind que aprendeu a jogar diversos jogos de videogame clássico (Atari) superando jogadores humanos, usando uma combinação de redes neurais profundas e aprendizado por reforço (o algoritmo conhecido como Deep Q-Network, DQN).

SAIBA MAIS

Agora que você viu exemplos práticos, vamos nos aprofundar um pouco mais nos fundamentos teóricos e nuances de cada paradigma de aprendizado de máquina, bem como discutir algumas extensões e variações importantes.

Aprendizado supervisionado – função, objetivo e generalização

Em problemas supervisionados, o objetivo fundamental é estimar uma função (modelo) que melhor aproxime a relação entre as variáveis de entrada e a saída esperada com base em exemplos históricos. Matematicamente, isso costuma ser formalizado como a minimização de uma função de perda (por exemplo, erro quadrático médio para regressão, entropia cruzada para classificação) sobre um conjunto de treinamento, estratégia conhecida como minimização do risco empírico.

Métodos como a regressão linear/logística, árvores de decisão, Random Forest, Support Vector Machines e redes neurais diferem em como representam essa função e no algoritmo utilizado para ajustar (treinar) os parâmetros a fim de minimizar a perda. Um aspecto crucial no aprendizado supervisionado é a generalização: um bom modelo não é aquele que acerta apenas os dados com que foi treinado, mas também consegue acertar futuras previsões em dados novos (que ele não viu durante o treino).

Técnicas como divisão em conjuntos de treino/validação/teste, validação cruzada e regularização são empregadas para estimar e melhorar a capacidade de generalização dos modelos. Além disso, é importante ficar atento(a) a problemas como overfitting (quando o modelo fica especializado demais nos exemplos de treinamento, perdendo desempenho em novos dados) e underfitting (quando o modelo é muito simples e não consegue capturar os padrões dos dados). Encontrar o equilíbrio adequado – normalmente aumentando a complexidade do modelo até o ponto ótimo antes de incorrer em overfitting – é parte da arte de construir modelos supervisionados eficazes.

Aprendizado não supervisionado – estrutura de dados e redução de dimensionalidade

Na ausência de rótulos, o aprendizado não supervisionado busca modelar a estrutura dos dados. Como observado no livro *Machine Learning: A Probabilistic Perspective*, de Kevin P. Murphy, esse paradigma pode ser entendido como um problema de estimação de densidade de probabilidade: em vez de modelar $P(y|x)$ (probabilidade de uma saída y dado entrada x , como no caso supervisionado), o foco está em modelar $P(x)$ – isto é, compreender a distribuição e as relações intrínsecas entre as variáveis de entrada.

Técnicas de aprendizado não supervisionado incluem algoritmos de agrupamento (como K-Means, DBSCAN, clustering hierárquico), redução de dimensionalidade (e.g., PCA – Análise de Componentes Principais) e modelos de detecção de anomalias, entre outros. Uma característica interessante do aprendizado não supervisionado é que seus resultados podem ser mais subjetivos de avaliar – afinal, sem rótulos “verdadeiros”, precisamos usar métricas indiretas (como silhouette score para avaliar coesão e separação de clusters, ou variância explicada para reduções de dimensionalidade) e a interpretação dos grupos ou fatores descobertos pode exigir conhecimento do domínio do problema.

Contudo, o aprendizado não supervisionado é poderoso para descobrir padrões ocultos, compressão de dados e inicialização de modelos. Por exemplo, algoritmos de agrupamento podem revelar segmentos de mercado em dados de clientes que não estavam óbvios à primeira vista e técnicas como PCA ou autoencoders podem extrair os fatores de variação mais relevantes dos dados, permitindo visualizar e compreender melhor estruturas complexas.

Vale notar também a existência do aprendizado semi-supervisionado – um meio-termo em que parte dos dados possui rótulos e parte não – e do aprendizado auto-supervisionado, uma forma de usar o próprio dado para gerar rótulos artificiais e treinar modelos preditivos (estratégia amplamente utilizada no treinamento de grandes modelos de linguagem, os Large Language Models, a partir de enormes quantidades de texto não rotulado).

Aprendizado por reforço – formalização e aprendizado profundo

O aprendizado por reforço se fundamenta em conceitos da teoria de controle e otimização sequencial de decisões. Formalmente, um problema de RL é frequentemente modelado como um Processo de Decisão Markoviano (Markov Decision Process, MDP), definido por um conjunto de estados S , um conjunto de ações A , uma função de recompensa $R(s,a)$ e possivelmente um modelo de transição de estados $T(s,a \rightarrow s')$ (no caso de RL baseado em modelo). O objetivo do agente é encontrar uma *política ótima $\pi(s)^*$, que define a melhor ação a tomar em cada estado s , de forma a maximizar a soma total de recompensas esperadas (também chamada de retorno).

Muitas abordagens de solução existem: algoritmos model-free como Q-Learning e SARSA aprendem estimativas das recompensas futuras (valores Q) para cada estado ou par estado-ação, enquanto métodos de gradiente de política aprendem diretamente uma política ótima, sem estimar explicitamente valores de estado. Com o advento do Aprendizado por Reforço Profundo (Deep RL), que combina redes neurais para aproximação de funções e algoritmos de RL, tornou-se possível enfrentar ambientes extremamente complexos e de alta dimensionalidade.

Essa combinação esteve por trás de avanços impressionantes, como o já citado AlphaGo, que em 2016 derrotou o campeão mundial de Go ao aprender estratégias por si só (algo que especialistas acreditavam estar a décadas de distância). Da mesma forma, técnicas de RL têm sido combinadas com modelos de linguagem natural: o ChatGPT, por exemplo, utiliza um método chamado aprendizado por reforço com feedback humano (Reinforcement Learning from Human Feedback, RLHF) para refinar suas respostas e alinhá-las melhor às preferências dos usuários.

Esses casos demonstram o potencial do aprendizado por reforço em ambientes complexos, mas também evidenciam desafios atuais: treinar agentes de RL em situações do mundo real (fora de contextos simulados ou jogos) ainda é difícil, devido à necessidade de enormes quantidades de interações para aprender com eficácia, ao desenho apropriado das funções de recompensa e às preocupações de segurança (por exemplo, como garantir que um robô aprendendo por reforço não cause danos no processo?).

Pesquisas em andamento buscam tornar o RL mais amostralmente eficiente e seguro, incluindo abordagens como exploração guiada, aprendizado por reforço multi-agente (vários agentes aprendendo em conjunto) e integrações do RL com aprendizado supervisionado/não supervisionado – por exemplo, usar modelos não supervisionados para extrair características do ambiente (representation learning) ou iniciar o agente com comportamentos imitando dados de especialistas (imitation learning).



MERCADO, CASES E TENDÊNCIAS

Os paradigmas de aprendizado de máquina discutidos até aqui estão na base de inúmeras inovações tecnológicas que vemos no mercado e continuam evoluindo rapidamente. Nesta seção, veremos casos de uso de cada paradigma e tendências atuais, incluindo como eles muitas vezes se combinam para resolver problemas complexos.

Por ser relativamente direto (dados rotulados $\Rightarrow$ predição), o aprendizado supervisionado tem aplicações consolidadas e de alto impacto em diversos setores. Na medicina, modelos supervisionados classificam imagens para diagnosticar doenças ou preveem riscos à saúde de pacientes. No setor financeiro, algoritmos supervisionados detectam fraudes em transações analisando padrões em dados históricos rotulados como “fraudulento” ou “legítimo”.

Em marketing, modelos preditivos identificam clientes propensos a cancelar um serviço ou respondem à pergunta “esse usuário vai clicar num anúncio?” com base em um histórico de interações rotulado. Esses são apenas alguns exemplos – sempre que há disponibilidade de dados históricos com respostas conhecidas, as técnicas supervisionadas podem ser empregadas para prever os resultados futuros ou classificar novos exemplos com alta acurácia.

Técnicas não supervisionadas também são amplamente utilizadas no mercado, especialmente para análise exploratória de dados e descoberta de conhecimento. Por exemplo, no varejo e comércio eletrônico, o agrupamento de clientes (clustering) é usado para descobrir segmentos de mercado e personalizar estratégias de marketing para grupos de clientes com comportamentos semelhantes. Em segurança da informação, algoritmos não supervisionados ajudam a detectar anomalias – por exemplo, identificando padrões incomuns de acesso ou transações potencialmente fraudulentas que se desviam significativamente do comportamento típico e podem indicar ataques ou fraudes.

Outra aplicação importante é na compressão de dados e redução de dimensionalidade: técnicas como PCA ou autoencoders (redes neurais autoencoder) condensam dados de alta dimensão em representações mais compactas, facilitando o armazenamento e processamento (essas técnicas são empregadas, por exemplo,

no processamento de imagens e sinais para reduzir ruído e destacar características importantes).

Uma tendência notável é que o aprendizado não supervisionado (em particular, o chamado aprendizado auto-supervisionado) tem sido usado para pré-treinar grandes modelos de IA. Os Large Language Models (LLMs) modernos – como o GPT-3 e seus sucessores – são treinados inicialmente de forma não supervisionada em conjuntos massivos de textos, aprendendo a representar a linguagem e prever palavras sem qualquer rótulo humano explícito. Esse conhecimento inicial amplo pode então ser transferido e refinado para tarefas específicas, muitas vezes combinando com aprendizado supervisionado ou por reforço nas etapas finais (como o RLHF no ChatGPT, citado anteriormente).

Graças a avanços em poder computacional e em algoritmos, o aprendizado por reforço deixou de ser uma curiosidade acadêmica e se tornou viável em aplicações de alto nível. Já mencionamos o marco histórico do AlphaGo (DeepMind) que, em 2016, derrotou o campeão mundial Lee Sedol; desde então, surgiram evoluções como AlphaZero (que aprende xadrez, shogi e Go partindo apenas das regras do jogo) e agentes que superaram profissionais em jogos complexos de estratégia em tempo real (StarCraft II) e cooperação (Dota 2).

Para além de jogos, o RL vem sendo aplicado em controle de processos industriais (otimização de resfriamento de data centers, controle de plantas químicas), logística e supply chain (roteirização dinâmica, alocação de recursos), recomendação (maximização de engajamento/valor de vida do cliente ao longo do tempo), finanças (alocação sequencial, market making com restrições de risco) e mobilidade/autonomia (navegação, decisão em tempo real).

Em cenários reais, frequentemente utiliza-se bandits/contextual bandits quando o problema é escolher ações com feedback imediato (por exemplo, qual promoção mostrar) e RL offline/sim-to-real quando não é seguro ou viável aprender explorando no ambiente de produção. Desafios práticos incluem definição de recompensa (alinhada ao negócio), exploração segura, parcial observabilidade, constrangimentos operacionais (SLA, segurança, compliance) e generalização.

O QUE VOCÊ VIU NESTA AULA?

Nesta primeira aula de Machine Learning Avançado, mapeamos os três paradigmas fundamentais — supervisionado, não supervisionado e por reforço — destacando que cada um se diferencia, sobretudo, pelo tipo de sinal/feedback disponível para treinar o modelo (rótulos conhecidos, ausência de rótulos ou recompensas em interação com um ambiente). Também combinamos fundamentos teóricos, intuições e exemplos práticos em Python (com bibliotecas como scikit-learn) para solidificar o entendimento e preparar você para aplicar os conceitos em problemas reais.

No aprendizado supervisionado, vimos como transformar dados rotulados em previsões e classificações de alta acurácia, passando por práticas essenciais de generalização (treino/teste, validação cruzada e regularização) e cuidados contra overfitting.

No aprendizado não supervisionado, exploramos como descobrir estruturas e padrões ocultos em dados sem rótulos, usando K-Means para segmentação (clustering) e reforçando o papel de PCA/autoencoders na redução de dimensionalidade e compressão (úteis em imagens, sinais e análise exploratória). Discutimos ainda o papel atual do auto-supervisionado como base de pré-treino de LLMs e como esse conhecimento é posteriormente refinado com abordagens supervisionadas ou até reforço com feedback humano para alinhar comportamento e qualidade de resposta.

Por fim, no aprendizado por reforço, formalizamos o problema com MDPs, discutimos o trade-off exploração vs. exploração e implementamos um bandit epsilon-greedy para mostrar, na prática, como o agente aprende preferências ao longo do tempo. Conectamos esses conceitos a casos de alto impacto (AlphaGo/AlphaZero, controle de processos, logística, recomendação e autonomia) e às tendências de combinar paradigmas (pré-treino não supervisionado + fine-tuning supervisionado + RL/HF), sempre com atenção a segurança, eficiência amostral e MLOps/LLMOps para produção.

Em síntese, você saiu desta aula com um mapa mental e a mão na massa sobre quando usar cada abordagem, como avaliar e operacionalizar modelos e como combiná-los para resolver problemas complexos de mercado.



REFERÊNCIAS

ARXIV. **Deep reinforcement learning from human preferences**. 2017. Disponível em: <https://arxiv.org/abs/1706.03741>. Acesso em: 12 fev. 2026.

ARXIV. **Training language models to follow instructions with human feedback**. 2022. Disponível em: <https://arxiv.org/abs/2203.02155>. Acesso em: 12 fev. 2026.

JMLR. **What Regularized Auto-Encoders Learn from the Data-Generating Distribution**. 2014. Disponível em: <https://jmlr.org/papers/volume15/alain14a/alain14a.pdf>. Acesso em: 12 fev. 2026.

JOLLIFFE, I. T. **Principal Component Analysis**. 2. ed. New York: Springer, 2002. Disponível em: <https://link.springer.com/book/10.1007/b98835>. Acesso em: 12 fev. 2026.

NATURE. **Human-level control through deep reinforcement learning**. 2015. Disponível em: <https://www.nature.com/articles/nature14236.pdf>. Acesso em: 12 fev. 2026.

NATURE. **Mastering the game of Go with deep neural networks and tree search**. 2016. Disponível em: <https://www.nature.com/articles/nature16961.pdf>. Acesso em: 12 fev. 2026.

NATURE. **Mastering the game of Go without human knowledge**. 2017. Disponível em: <https://www.nature.com/articles/nature24270.pdf>. Acesso em: 12 fev. 2026.

OPENAI. **Aligning language models to follow instructions**. 2022. Disponível em: <https://openai.com/index/instruction-following/>. Acesso em: 12 fev. 2026.

SUTTON, R. S.; BARTO, A. G. **Reinforcement Learning: An Introduction**. 2. ed. Cambridge: MIT Press, 2018. Disponível em: <https://mitpress.mit.edu/9780262039246/reinforcement-learning/>. Acesso em: 12 fev. 2026.

TAYLOR & FRANCIS. **Zero-inflated models — Knowledge and References**. 2026. Disponível em: https://taylorandfrancis.com/knowledge/Engineering_and_technology/Engineering_support_and_special_topics/Zero-inflated_models. Acesso em: 12 fev. 2026.

WIKIPEDIA. **Principal component analysis**. 2026. Disponível em: https://en.wikipedia.org/wiki/Principal_component_analysis. Acesso em: 12 fev. 2026.

WIKIPEDIA. **Zero-inflated model**. 2026. Disponível em: https://en.wikipedia.org/wiki/Zero-inflated_model. Acesso em: 12 fev. 2026.

PALAVRAS-CHAVE

Aprendizado Supervisionado. Aprendizado não supervisionado. Aprendizado por Reforço.

EMAP



POSTECH